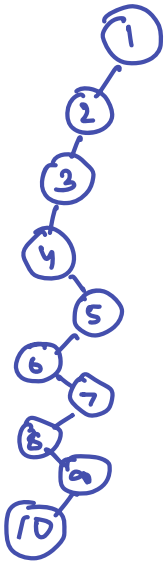
2.1



Let's say we have a min heap

$n = 1000$ so $\log n = 10$ we will have total 10 levels

now considering the property of heap root ≤ child

so smallest element we can have at last level is

10

2.2 For n elements there are $\log n$ levels at each level total number of nodes $= 2^{level}$

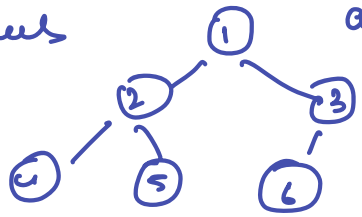
So total number of nodes at $l-1$ level 2^{l-1}

so all nodes until level $l-1$ are $2^{(l-1)+1} - 1$

$= 2^l - 1$

so last level will have $n - (2^l - 1)$ leaf nodes.

Eg tree with 3 levels and 6 nodes



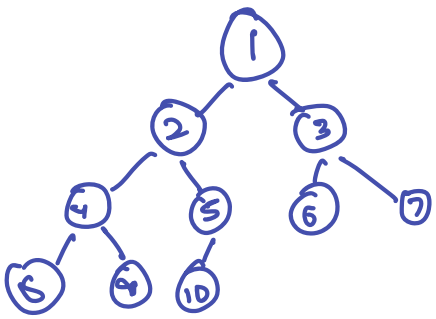
height : 3

level 1 has $2^1 = 2$

total till level 1 $= 2^{1+1} - 1 = 4 - 1 = 3$

now total leaf nodes : $6 - 3 = 3$

2.3 heap has $1 \rightarrow n$ lets say it has numbers $1 \rightarrow 10$

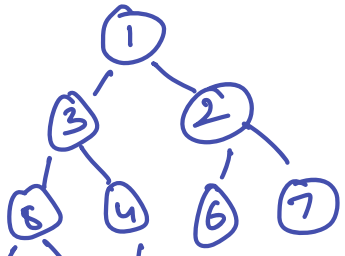


remove - min in this case removes 1

put 10 at root and sift down

takes max time $O(\log(n))$

for least amount of time



in this case we do it in $O(1)$ because 5 becomes root and then needs exactly 1 sift down

2.4 for 2 children $2 \cdot i + 1$ & $2 \cdot i + 2$

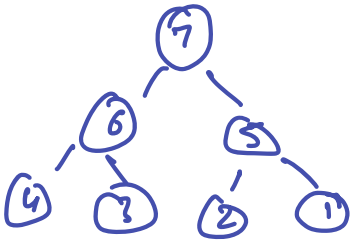
so for 3 children the indices would be $2 \cdot i + 1$, $2 \cdot i + 2$ & $2 \cdot i + 3$

2.8 for $i = 0 \ldots n-1$

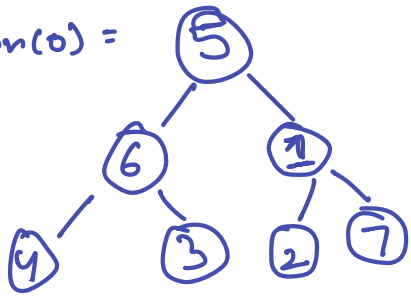
sift-down(i)

let's take an array

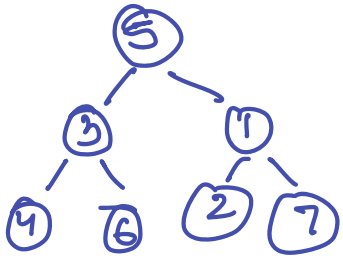
7, 6, 5, 4, 3, 2, 1



sift-down(0) =



sift down (1) :



we already break the heap property of root ≤ child so this algorithm doesn't work.